Teamify – návrh architektúry

Konverzácia Lukáš × Claude · 29. 10. 2026 · priebežné poznámky: Teamify – poznámky k architektúre

Lukáš

Počas tejto diskusie si píš poznámky. Nemusí to byť záznam nášho rozhovoru v mierke 1:1, ale aby tam boli podstatné informácie ohľadom tejto konverzácie.

Claude

Jasné, budem si viesť priebežné poznámky. Dám ich do Claude Doc, ktorý založím hneď, ako začneme riešiť samotnú tému, a po každej podstatnejšej časti ho doplním. Pôjde o zhrnutie: kľúčové informácie, rozhodnutia, otvorené otázky a ďalšie kroky, nie prepis rozhovoru. Doc môžeš kedykoľvek otvoriť, upraviť alebo stiahnuť ako Word či PDF.

O čom ideme diskutovať?

Lukáš

Príloha: Teamify.txt (popis systému a 31 user stories)

ROLA Si skúsený softvérový architekt, ktorý vysvetľuje veci jasne a úprimne, vrátane nevýhod.

CIEĽ Pomôž mi pochopiť rôzne architektúry, ktorými by sa dal implementovať systém opísaný v priloženom súbore s požiadavkami.

SITUÁCIA Ide o univerzitné zadanie. Do chatu ti prikladám súbor s požiadavkami na softvér, ktorý mi bol pridelený.

OČAKÁVANÝ VÝSLEDOK 3 naozaj odlišné architektúry pre tento systém, ich trade-offy a náročné časti implementácie.

KROKY

1. Prečítaj si priložený súbor.
2. Stručne zhrň, čo systém robí, a jeho kľúčové požiadavky.
3. Potom sa zastav a počkaj na moje otázky. Architektúry nenavrhuj, kým ťa o to nepožiadam.

Claude

(Založené poznámky: Teamify – poznámky k architektúre)

Teamify v skratke: aplikácia na osobnú správu úloh a tímovú spoluprácu. Každý používateľ má osobnú agendu. Okrem nej môže byť v tímoch, ktoré majú zdieľanú agendu udalostí, skupinový chat a ankety. Celé to spájajú notifikácie v reálnom čase.

Kľúčové požiadavky (31 user stories):

- **Účet (1–7):** registrácia, prihlásenie a odhlásenie, zobrazenie a úprava profilu, reset hesla, zmazanie účtu.
- **Osobné úlohy (9, 11–14):** agenda, vytváranie, úprava, mazanie a označenie splnenia.
- **Tímy a roly (15–20, 25–26):** vytvorenie, pripojenie a opustenie tímu, úprava a zmazanie tímu. Admin môže povýšiť ďalších adminov alebo im práva odobrať.
- **Tímové udalosti (10, 27–31):** zdieľaná agenda. Udalosti vytvára a spravuje admin, členovia prijímajú alebo odmietajú účasť.
- **Komunikácia (8, 21):** sekcia notifikácií a skupinový chat v reálnom čase.
- **Ankety (22–24):** admin ich vytvára, členovia hlasujú a vidia výsledky.

Čo je architektonicky podstatné:

- Systém kombinuje bežné CRUD operácie s real-time komunikáciou: server musí vedieť sám poslať správu klientovi.
- Oprávnenia závisia od roly v konkrétnom tíme, a tá sa môže meniť za behu.
- Zadanie nechává otvorené niekoľko vecí: ako presne funguje pripojenie k tímu (pozvánka, žiadosť alebo kód), čo sa stane, keď tím opustí posledný admin, a čo sa stane s dátami pri zmazaní účtu.

Zhrnutie je aj v poznámkach. Teraz čakám na tvoje otázky.

Lukáš

navrhni 3 architektúry, ktorými by sa dal implementovať tento systém

Claude

Tu sú tri naozaj odlišné prístupy. Detaily, nevýhody a porovnávacia tabuľka sú v poznámkach.

A: Modulárny monolit. Jedna aplikácia, vnútri rozdelená na moduly (účty, tímy, úlohy, chat, ankety, notifikácie), nad jednou relačnou databázou. Klient volá REST a pre chat a notifikácie drží WebSocket spojenie. Je to najjednoduchšie a transakcie fungujú bez problémov. Nevýhoda: škáluje sa len celý systém naraz, a hranice medzi modulmi treba aktívne strážiť, inak sa rozmažú.

B: Mikroslužby s event busom. Každá oblasť je samostatná služba s vlastnou databázou. Služby si posielajú udalosti cez broker (RabbitMQ alebo Kafka) a notifikačná služba ich preposiela klientom. Každá časť sa dá škálovať zvlášť. Cena je vysoká: veľa infraštruktúry, ťažké ladenie a eventual consistency. Napríklad zmazanie tímu sa musí rozbehnúť cez viacero služieb.

C: Serverless / BaaS (Firebase, Supabase). Žiadny vlastný server. Autentifikácia, databáza aj real-time odbery sú hotové, vlastnú logiku píšeš ako malé cloudové funkcie. Je to najrýchlejšie na vývoj, ale prináša vendor lock-in. Oprávnenia sa navyše píšu ako databázové pravidlá, ktoré sa ťažko testujú. A vyučujúci môže namietať, že ste architektúru nenavrhli, len prevzali.

Úprimne: na univerzitné zadanie by som volil A. B má zmysel, len ak je cieľom predmetu práve distribuovaný systém.

V každej z troch architektúr budú náročné tie isté miesta: oprávnenia podľa roly v konkrétnom tíme (tím nesmie ostať bez admina), notifikácie pre používateľov, ktorí sú práve offline, mazanie účtu alebo tímu aj so všetkými naviazanými dátami, a agenda, ktorá spája osobné úlohy s udalosťami zo všetkých tímov.

Chceš niektorú z nich rozobrať hlbšie, napríklad nakresliť diagram komponentov?

Lukáš

Nechápem úplne, ako klient volá ten REST a prečo sa tie chat notifikácie držia vo websockete. Vie to fungovať aj offline, keď sa používajú webové služby, alebo funguje to iba keď je online? Respektíve je nutné, aby ten systém fungoval aj offline? Potom k tomu B-čku, mať vlastnú databázu ku každej službe mi príde strašne náročné. Myslíš si, že to je najlepší spôsob, ako to spraviť? A tie udalosti cez broker, neviem, čo to znamená, prosím ťa, vysvetli mi to. RabbitMQ a Kafka netuším, vysvetli mi to lepšie. A ako by fungoval v tom C-čku serverless, keď ideálne potrebujem komunikáciu medzi jednotlivými členmi tímu? Príde mi, že sa to nedá použiť, keď to je serverless, že by to muselo byť asi iba v rámci firmy, v ktorej pracujeme. Či? Prosím ťa, priblíž mi to viac.

Claude

Poďme po poradí.

1. REST a WebSocket

REST funguje ako bežná webová požiadavka. Klient (aplikácia v prehliadači alebo v mobile) pošle na server požiadavku cez internet, napríklad „daj mi moje úlohy" (`GET /tasks`) alebo „vytvor úlohu" (`POST /tasks`). Server odpovie a tým to končí. Iniciatívu má vždy klient: server sám od seba nemôže nič poslať, iba odpovedá.

Pri chate to nestačí. Keď ti kolega pošle správu, server ti ju potrebuje doručiť hneď, ale ty si sa práve na nič nepýtal. Mohol by si sa pýtať každé dve sekundy („je niečo nové?"), lenže to je plytvanie. WebSocket preto drží medzi klientom a serverom otvorené spojenie. Server ním môže kedykoľvek „zatlačiť" novú správu alebo notifikáciu.

Analógia: REST je ako písať e-maily, keď sa niečo pýtaš. WebSocket je ako telefonát, ktorý nezložíš, takže druhá strana môže prehovoriť kedykoľvek.

2. Offline

Nie, zadanie offline režim nevyžaduje. Chat a notifikácie v reálnom čase predpokladajú, že si pripojený na internet. Treba ale vyriešiť používateľa, ktorý práve nie je prihlásený. Notifikácia sa mu uloží do databázy a uvidí ju v sekcii notifikácií, keď sa prihlási (user story 8). Plný offline režim, kde by si pracoval bez internetu a dáta by sa neskôr zosynchronizovali, je veľká extra práca s riešením konfliktov. Do zadania by som ho nedával.

3. Databáza pre každú službu

Máš dobrý inštinkt, pre tento projekt to najlepší spôsob nie je. V „učebnicových" mikroslužbách sa to robí preto, aby služby neboli zviazané cez spoločné tabuľky. Ak by chat čítal priamo tabuľku tímov, zmena tej tabuľky by rozbila aj chat a služby by už neboli nezávislé. Lenže práve z toho vyplýva tá náročnosť, ktorú vidíš. Rozumný kompromis je jeden databázový server s oddelenými schémami, pričom každá služba siaha len na tú svoju. A úprimne, ak vás predmet nenúti robiť mikroslužby, celá architektúra B je pre Teamify zbytočne ťažká.

4. Broker, RabbitMQ a Kafka

Broker je sprostredkovateľ správ medzi službami, niečo ako pošta. Bez neho by služba Udalosti po vytvorení novej udalosti musela priamo volať notifikačnú službu, potom ďalšiu a ďalšiu, a keby jedna z nich práve nebežala, má problém. S brokerom iba „hodí list na poštu": *v tíme X vznikla udalosť Y*. Pošta ho doručí každému, kto sa o takéto správy prihlásil, a ak je adresát práve nedostupný, počká naň.

- **RabbitMQ** je klasická pošta s frontami. Správa čaká vo fronte, kým ju niekto spracuje, a potom zmizne.
- **Kafka** je skôr trvalý denník udalostí. Správy v ňom ostávajú a každý čitateľ si ho číta vlastným tempom, pokojne aj od začiatku znova.

Pre Teamify by stačil RabbitMQ. Kafka je stavaná na obrovské objemy dát.

5. Serverless

Tu je časté nedorozumenie. Serverless neznamená, že server neexistuje, ani že systém funguje len vo firme. Servery existujú, ale nespravuješ ich ty: beží to v cloude u poskytovateľa (Google pri Firebase, Supabase, AWS). Aplikácia funguje cez internet pre kohokoľvek a kdekoľvek, presne ako bežná webová aplikácia.

Chat by fungoval takto: Jano napíše správu a jeho aplikácia ju zapíše do cloudovej databázy. Aplikácie ostatných členov tímu majú na tú databázu „odber", takže ich databáza sama upozorní a správa sa im okamžite zobrazí. Ten real-time push, ktorý v architektúre A robíš cez vlastný WebSocket, tu dostaneš hotový od poskytovateľa. Komunikácia medzi členmi teda funguje bez problémov. Ide len o to, kto sa stará o servery.

Ak chceš, môžem ti nakresliť diagram, ako presne putuje jedna správa v chate pri každej z troch architektúr. Na tom sa rozdiely ukážu najlepšie. Vysvetlenia som doplnil aj do poznámok.

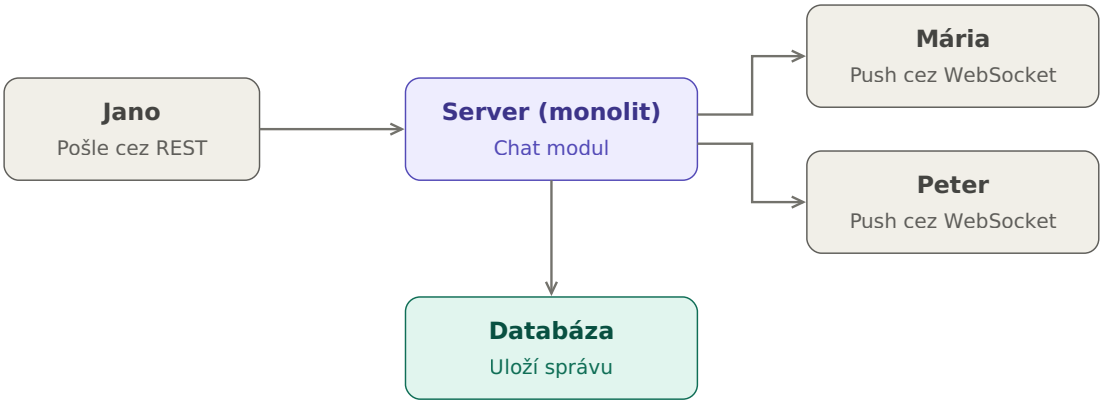
Lukáš

ano nakresli mi to

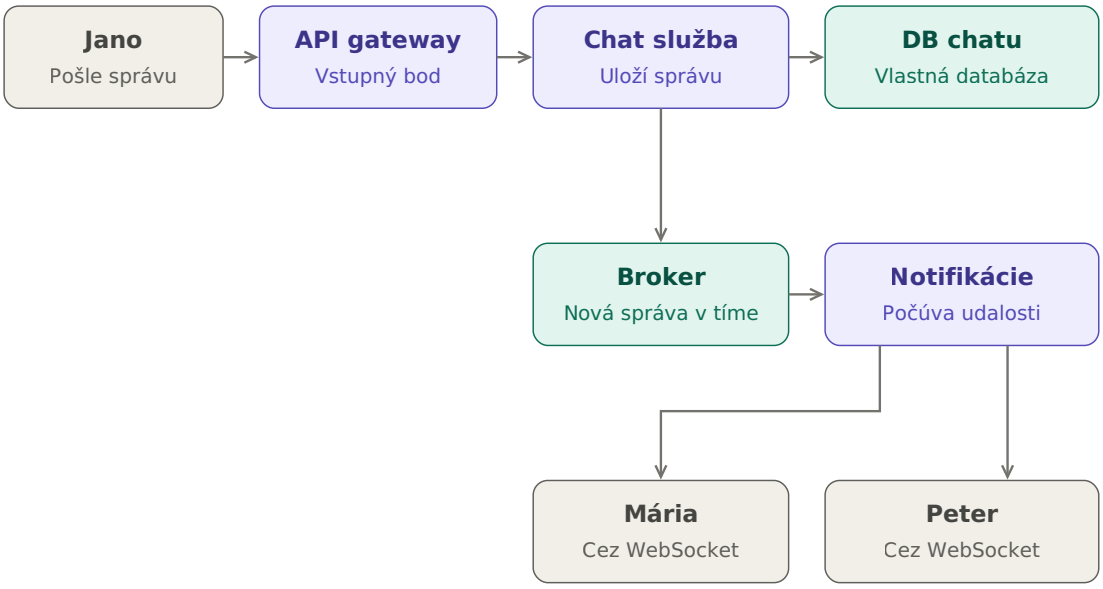
Claude

Jano pošle do tímového chatu správu „Stretko je o 18:00". Takto sa správa dostane k Márii a Petrovi v jednotlivých architektúrach. Šedé sú aplikácie používateľov, fialové kód, ktorý píšete vy, a tyrkysové dáta alebo infraštruktúra.

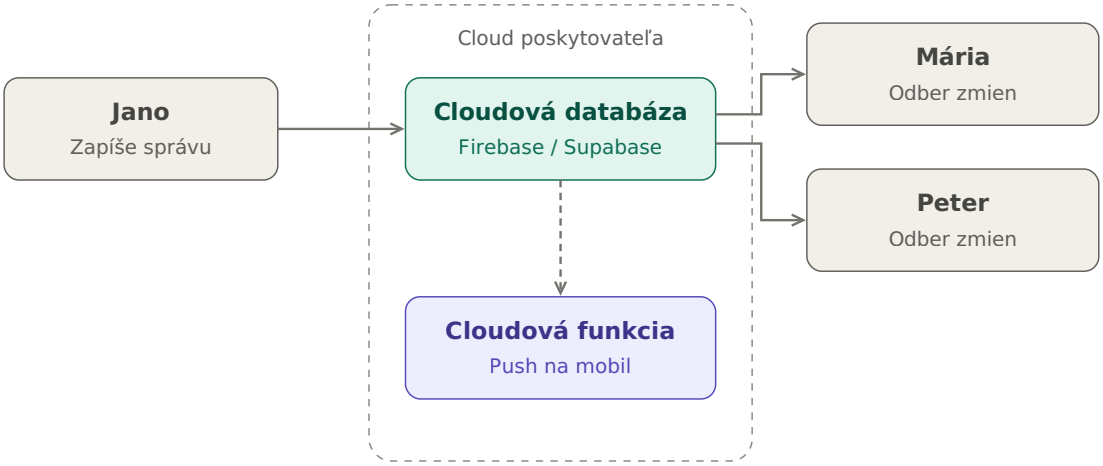
A: Modulárny monolit. Všetko ide cez jeden server. Jano pošle správu cez REST, chat modul ju uloží do databázy a hneď ju cez otvorené WebSocket spojenia „zatlačí" ostatným členom tímu.



B: Mikroslužby. Chat služba správu iba uloží a ohlási brokeru, že vznikla nová správa. O doručenie sa postará samostatná notifikačná služba, ktorá takéto udalosti počúva a drží WebSocket spojenia. Služby o sebe priamo nevedia a ich jediným spojením je broker.



C: Serverless / BaaS. Vlastný server tu nie je, všetko v prerušovanom rámčeku prevádzkuje poskytovateľ v cloude. Jano zapíše správu priamo do cloudovej databázy. Aplikácie Márie a Petra majú na tú databázu odber, takže im ju databáza sama pošle. Cloudová funkcia je voliteľná, napríklad na push notifikáciu do mobilu, keď má niekto aplikáciu zavretú.



Keď diagramy porovnáš vedľa seba, uvidíš hlavný rozdiel. V A si celú cestu správy staviate sami, ale na jednom mieste. V B ju tiež staviate sami, no rozsekanú na viac častí. V C dostanete real-time doručenie hotové a píšete len to, čo je okolo neho.

Postup som zapísal aj do poznámok. Ak chceš, môžem rovnako rozkresliť aj zložitejší scenár, napríklad keď admin vytvorí udalosť a všetci členovia dostanú pozvánku.

Lukáš

Dobre, a teraz mi vysvetli, čo bude najväčší pain implementovať v low-level implementácii, a aké trade-offy budeme musieť spraviť v jednotlivých architektúrach.

Claude

Najprv to dôležité: najťažšie nebudú obrazovky ani CRUD operácie (vytvor, uprav, zmaž úlohu). Najviac vás potrápia tri veci: doručovanie v reálnom čase, súbežnosť (dvaja ľudia robia to isté naraz) a pravidlá, ktoré musia platiť vždy, napríklad že tím nesmie ostať bez admina.

A: Modulárny monolit

Najväčší pain: správa WebSocket spojení. Na papieri je to „otvor spojenie a posielaj správy", v kóde musíš vyriešiť oveľa viac:

- Server si musí pamätať, ktoré spojenie patrí ktorému používateľovi. Jeden človek môže mať otvorené tri: mobil, notebook a druhú kartu v prehliadači.
- Spojenie musí byť overené. Kto sa pripája, preukáže sa tokenom už pri nadviazaní spojenia.
- Keď na sekundu vypadne Wi-Fi, spojenie spadne a klient sa musí sám znova pripojiť. Správy, ktoré medzitým prišli, by sa stratili. Klient si preto musí pamätať ID poslednej prijatej správy a po pripojení si cez REST vypýtať, čo zmeškal.

Ďalšie bolesti:

- **Oprávnenia:** každý endpoint musí overiť, či si v tíme a či si admin. Stačí na to raz zabudnúť a máš bezpečnostnú dieru. Riešením je jedno centrálne miesto (middleware alebo policy), nie kontroly roztrúsené po kóde.
- **Race condition pri adminoch:** tím má dvoch adminov a obaja v tej istej sekunde odoberú práva tomu druhému. Obe kontroly „ostane aspoň jeden admin?" prejdú, lebo každá vidí ešte toho druhého, a tím ostane bez admina. Treba to robiť v transakcii so zamknutím riadkov.

Trade-off: dostanete jednoduchosť a transakcie zadarmo, platíte škálovaním. Kým beží jeden server, všetko funguje. Pri dvoch serveroch sa môže stať, že Jano je pripojený na prvý a Mária na druhý, a prvý server o Márii nevie. Vtedy treba pridať Redis pub/sub, aby si servery správy preposielali.

B: Mikroslužby

Najväčší pain: dual write. Chat služba musí urobiť dve veci: uložiť správu do svojej databázy a poslať udalosť do brokera. Čo ak sa prvá podarí a pred druhou služba spadne? Správa je uložená, ale nikto sa o nej nedozvie. Opačne je to ešte horšie. Riešenie sa volá outbox pattern: udalosť sa zapíše do špeciálnej tabuľky v tej istej transakcii ako správa a samostatný proces ju odtiaľ posiela do brokera. Funguje to, ale je to kód navyše, ktorý v A vôbec nepotrebuješ.

Ďalšie bolesti:

- **Zastarané oprávnenia:** služby nevidia tabuľku tímov, takže rolu používateľa väčšinou čítajú z JWT tokenu. Keď niekomu odoberieš admina, jeho token o tom nevie, kým nevyprší. Bývalý admin môže ešte pár minút mazať udalosti.
- **Duplicity a poradie:** broker zaručuje doručenie „aspoň raz", nie „presne raz". Tá istá udalosť môže prísť dvakrát a správy môžu doraziť v inom poradí. Každý konzument musí byť idempotentný, teda spracovanie tej istej udalosti dvakrát nesmie nič pokaziť.
- **Mazanie tímu:** mazanie je rozložené cez viac služieb (udalosti, ankety, chat). Ak jedna z nich zlyhá, zvyšok treba vrátiť alebo dokončiť neskôr. To je takzvaná saga.
- **Ladenie:** jedna akcia prechádza cez štyri služby, takže na hľadanie chýb potrebuješ korelačné ID v logoch a celý systém spustený v Docker Compose.

Trade-off: dostanete nezávislosť služieb, platíte konzistenciou. Systém je istý čas „trochu nepravdivý". Napríklad Peter odíde z tímu, ale chat služba sa to dozvie o chvíľu neskôr, takže mu ešte príde jedna správa.

C: Serverless

Najväčší pain: pravidlá, ktoré sa nedajú dobre napísať v security rules. Pravidlo „udalosť môže vytvoriť iba admin" sa v nich zapíše ľahko. Pravidlo „admin si nemôže odobrať práva, ak je posledný" je problém, lebo pravidlo by muselo spočítať adminov v tíme, a to rules robia zle alebo vôbec. Takú operáciu musíš presunúť do cloudovej funkcie a klientovi zakázať priamy zápis. Postupne tak zistíš, že kus „bezserverovej" aplikácie je vlastne server rozsekaný na funkcie.

Ďalšie bolesti (hlavne pri Firestore, teda NoSQL):

- **Agenda:** osobné úlohy plus udalosti zo všetkých tímov znamenajú viac dotazov, ktoré sa skladajú v klientovi. JOIN tu nemáš.
- **Denormalizácia:** názov tímu sa kopíruje do každej udalosti. Keď admin tím premenuje, treba ho prepísať všade.
- **Hlasovanie:** keď päť ľudí naraz zvýši počítadlo hlasov, bez transakcie sa niektoré hlasy stratia.
- **Mazanie:** zmazanie dokumentu nezmaže jeho podkolekcie, takže tím zmažeš, ale jeho chat ostane v databáze ako sirota.

Supabase (Postgres s row-level security) väčšinu z toho zjemní, lebo má JOINy, transakcie a kaskádové mazanie. Pravidlá posledného admina však budú náročné aj tam.

Trade-off: dostanete rýchlosť vývoja, platíte kontrolou nad systémom a závislosťou od poskytovateľa.

Spoločné pre všetky tri

- Proti dvojitému hlasovaniu daj do databázy unikátne obmedzenie na dvojicu (anketa, používateľ). Neriešiť to iba v kóde.
- Pri zmazaní účtu správy v chate nemaž, len autora nahraď textom „Zmazaný používateľ". Inak by z konverzácií zostali diery.
- Časy ukladaj v UTC a na miestny čas ich prevádzaj až v klientovi.

Celé je to zhrnuté v tabuľke v poznámkach. Ak chceš, môžeme niektorý z tých problémov rozobrať až na úroveň kódu, napríklad race condition s adminmi alebo outbox pattern.

Lukáš

OK, dobre, teraz chápeme, čo bude ťažké implementovať a čo nebude. Tak podľa týchto informácií, ktoré si mi práve dal, navrhni, ktorá architektúra je najjednoduchšia, ale zároveň aj funkčná na implementáciu.

Claude

Odporúčam **A, modulárny monolit**. Je najjednoduchší a zároveň pokryje všetkých 31 user stories.

Prečo práve on:

- Väčšina toho, čo sme označili za ťažké, v ňom vôbec nevznikne. Nepotrebuješ outbox pattern ani sagy, nehrozia zastarané oprávnenia v tokenoch a duplicitné správy z brokera neriešiš, lebo broker nemáš.
- To, čo v serverless robí problémy (posledný admin, kaskádové mazanie, počítadlo hlasov), tu vyrieši bežná transakcia a obmedzenia v relačnej databáze.
- Ostanú dve ťažšie veci: WebSocket spojenia (opätovné pripojenie a dotiahnutie zmeškaných správ) a race condition pri adminoch. Obe sú dobre známe problémy so zavedeným riešením.

Ako si to ešte zjednodušiť:

- Jedna inštancia servera. Bez Redisu, bez load balancera. Pre školský projekt to úplne stačí.
- Notifikácie spracuj jedným spôsobom pre všetkých: vždy ich ulož do databázy a zároveň pošli cez WebSocket. Kto je online, dostane ich hneď, kto nie je, uvidí ich po prihlásení. Nemusíš riešiť dva rôzne prípady.
- Pripojenie k tímu cez pozývací kód (admin ho vygeneruje a pošle ostatným) je najjednoduchší výklad nejasnej user story 16.
- Stack nechaj na to, čo už ovládate. Funguje napríklad Spring Boot alebo NestJS či Django, k tomu PostgreSQL a React. Všetky tri backendy majú hotovú podporu WebSocketov.

Kedy by som volil inak: ak by predmet nehodnotil návrh backendu a išlo by hlavne o to, mať rýchlo funkčnú aplikáciu, zvolil by som C v podobe Supabase. Firestore nie, lebo Supabase je relačná databáza a ušetrí vás väčšiny NoSQL problémov.

Odporúčanie som doplnil do poznámok. Ak chceš, ako ďalší krok ti môžem rozkresliť moduly monolitu a ich prepojenia, alebo navrhnúť databázovú schému.